COMPUTER ARCHITECTURE

Module 5: Multiprocessor Architecture

Comprehensive Question & Answer Notes · 5-7 Mark Level · Even Semester

Author: Soumya Chakraborty · 26 Questions

Q1 Define Flynn's Taxonomy. List the four categories.

Flynn's Taxonomy (proposed by Michael J. Flynn in 1966) is the most widely used classification system for computer architectures. It categorizes systems based on the number of concurrent **instruction streams** and **data streams** they can process.

The Four Categories:

Category	Instruction Streams	Data Streams	Description	Example
SISD	Single	Single	Traditional uniprocessor. One instruction operates on one data item at a time. Sequential execution.	Classic Pentium, older microcontrollers
SIMD	Single	Multiple	One instruction is broadcast to multiple processing elements, each operating on different data. Data-level parallelism.	GPUs, Vector processors (Cray), Intel AVX/SSE
MISD	Multiple	Single	Multiple instructions operate on the same data stream. Extremely rare in practice.	Fault-tolerant systems (Space Shuttle flight computers), systolic arrays (debatable)
MIMD	Multiple	Multiple	Multiple processors execute different instructions on different data independently. Most flexible and powerful.	Multi-core CPUs (Intel i9), Clusters, Distributed systems

Significance: While simplified (it doesn't capture nuances like multi-threading or GPU architectures well), Flynn's Taxonomy remains the standard introductory framework for discussing parallel computing architectures.

Q2 What is the difference between UMA and NUMA?

UMA and NUMA are two models of shared-memory multiprocessor architectures, differing in how memory access latency is distributed.

Feature	UMA (Uniform Memory Access)	NUMA (Non-Uniform Memory Access)
Memory Access Time	All processors have **equal access time** to all memory locations. Every memory address takes the same time regardless of which processor requests it.	Access time **depends on the location** of memory relative to the processor. Local memory is fast; remote memory (on another node) is slower.
Architecture	All processors share a single, centralized main memory connected via a common bus or crossbar switch.	Memory is physically distributed across nodes. Each processor has its own local memory, but can access other processors' memory via an interconnection network.
Scalability	Poor. The shared bus/crossbar becomes a bottleneck beyond ~8-16 processors.	Good. Can scale to hundreds of processors because most accesses are to local memory, reducing bus contention.
Programming	Simpler. Programmers don't need to worry about data placement.	More complex. Programmers must optimize data locality to minimize remote memory accesses for good performance.
Examples	Small SMP servers, Intel single-socket systems	AMD EPYC, Intel Xeon multi-socket servers, SGI Origin

Key Insight: NUMA is essentially a compromise between shared-memory ease-of-programming and distributed-memory scalability. The programmer sees a single shared address space, but performance depends on data placement.

Q3 What do you mean by memory consistency?

Memory Consistency (or Memory Consistency Model) defines the rules governing the **order in which memory operations** (reads and writes) performed by one processor become visible to other processors in a shared-memory multiprocessor system.

Why It Matters:

Modern processors reorder instructions for performance (out-of-order execution, write buffers, store-to-load forwarding). In a single processor, this is invisible because the hardware preserves the illusion of sequential execution. In a multiprocessor, however, reordering can cause one processor to see another processor's memory operations in a different order than they were issued, leading to bugs in parallel programs.

Common Consistency Models:

- **Sequential Consistency (SC):** The strictest model (defined by Lamport). The result of any execution is the same as if the operations of all processors were interleaved in some sequential order, and each processor's operations appear in the order specified by its program. Simple to reason about but restricts hardware optimizations.
- **Total Store Order (TSO):** Used by x86 processors. Allows a processor to read its own write before the write is globally visible (store buffer forwarding), but all other ordering guarantees are maintained.
- **Relaxed/Weak Consistency:** Used by ARM and RISC-V. Allows extensive reordering of memory operations for maximum performance. Explicit **memory fence/barrier** instructions are required by programmers to enforce ordering when needed (e.g., before releasing a lock).

Q4 What is a dataflow computer?

A **Dataflow Computer** is a non-Von Neumann architecture where instruction execution is determined entirely by the **availability of input data (operands)**, rather than by a sequential Program Counter.

Key Principles:

- **No Program Counter:** Unlike Von Neumann machines, there is no global PC that fetches instructions sequentially. Instead, instructions are represented as nodes in a directed graph (the Dataflow Graph).
- **Data-Driven Execution:** An instruction "fires" (executes) as soon as ALL of its input operands arrive on its input arcs via "tokens." Results are sent as tokens along output arcs to dependent instructions.
- **Implicit Parallelism:** Because all instructions with available data can execute simultaneously, the hardware naturally discovers and exploits the maximum parallelism inherent in the program without any explicit scheduling.
- **No Side Effects:** Pure dataflow does not have global memory or variables. Data flows through the graph, making programs inherently free of data races.

Advantages: Naturally parallel, no hazards from sequential assumptions, excellent for functional programming paradigms.

Disadvantages: High overhead for token matching hardware, poor performance for sequential algorithms, difficulty handling irregular control flow and data structures. These limitations prevented widespread commercial adoption.

Q5 Give one example of a systolic array application.

Matrix Multiplication is the most classic and widely cited application of systolic arrays.

Why Systolic Arrays Excel at Matrix Multiply:

- Matrix multiplication involves highly regular, repetitive computations: multiply and accumulate (MAC) operations.
- Data flows in predictable, rhythmic patterns — perfect for the "pump-like" data flow of systolic arrays.
- Each Processing Element (PE) performs the same operation (multiply and add), and data is reused as it flows through neighboring PEs, minimizing expensive memory accesses.

Other Applications:

- **2D Convolution (Image Processing):** Applying a kernel/filter to an image — each PE holds one weight of the filter.
- **DNA/Protein Sequence Alignment:** Dynamic programming algorithms like Smith-Waterman map beautifully to systolic arrays.
- **Neural Network Inference:** Google's TPU (Tensor Processing Unit) uses a large 256×256 systolic array specifically for matrix multiply operations in deep learning.
- **Signal Processing / FFT:** FIR filters and Fourier transforms.

Q6 What is a reduction computer?

A **Reduction Computer** is a non-Von Neumann architecture based on **demand-driven (lazy) evaluation**. Instead of executing instructions eagerly from top to bottom, a reduction computer only evaluates an expression when its result is explicitly **needed** by another computation.

Key Concepts:

- **Demand-Driven Execution:** Computation starts from the "output" (the final result needed) and works backward, recursively demanding the sub-results needed to compute it. An expression is only evaluated ("reduced") when its value is required.
- **Graph Reduction:** The program is represented as an expression tree or graph. Execution proceeds by repeatedly selecting a "reducible expression" (redex) and replacing it with its value, until the entire tree is reduced to a single value.
- **Functional Programming Fit:** Reduction machines are designed to efficiently execute functional programming languages (like Haskell or Miranda), where programs are pure mathematical expressions without side effects.

Difference from Dataflow:

- **Dataflow:** Data-driven (eager). Executes as soon as data is available (push model).
- **Reduction:** Demand-driven (lazy). Executes only when the result is demanded (pull model).

Example: If the program computes `if True then X else (expensive_computation)`, a reduction machine never evaluates `expensive_computation` because the `else` branch is never demanded.

Q7 Define synchronization in parallel computing.

Synchronization in parallel computing refers to the mechanisms and protocols used to coordinate the execution of multiple concurrent threads or processes to ensure **correct program behavior** when they access shared resources.

Why It's Needed:

When multiple processors/threads access shared data concurrently, without synchronization, **race conditions** can occur — the final result depends on the unpredictable timing of operations, leading to incorrect or inconsistent outcomes.

Types of Synchronization:

1. **Mutual Exclusion (Locks/Mutexes):** Ensures that only one thread at a time can access a critical section (shared data). Implemented using atomic hardware instructions like `Test-and-Set` or `Compare-and-Swap (CAS)`.
2. **Barriers:** A synchronization point where all threads must arrive before any of them can proceed. Used to ensure all threads have completed a phase of computation before starting the next phase.
3. **Semaphores:** A generalized synchronization mechanism using a counter. Binary semaphores behave like mutexes. Counting semaphores can allow up to N threads to access a resource simultaneously.
4. **Condition Variables:** Allow threads to wait for a specific condition to become true, and be notified when another thread changes the condition.

Hardware Support: Modern CPUs provide atomic instructions (Test-and-Set, Load-Linked/Store-Conditional, Compare-and-Swap) that guarantee indivisible read-modify-write operations on memory, forming the foundation for all software synchronization primitives.

Q8 What is a distributed shared memory (DSM) architecture?

Distributed Shared Memory (DSM) is an architecture (implemented in hardware, software, or both) that presents a **single, unified shared address space** to the programmer, even though the underlying physical memory is **distributed across multiple separate nodes** in a network.

How It Works:

- Each node has its own processor and local physical memory.
- The DSM system maps all local memories into a single global virtual address space.
- **Local Access:** When a processor accesses an address that resides in its own local memory, the access is fast (same as a regular memory access).
- **Remote Access:** When a processor accesses an address that resides in another node's physical memory, the DSM layer transparently fetches the data over the network. This is slower (NUMA effect).

Consistency Maintenance:

- **Hardware DSM (e.g., SGI Origin, AMD ccNUMA):** Uses directory-based cache coherence protocols to track which caches hold copies of each memory block and ensure consistency.
- **Software DSM (e.g., TreadMarks, Ivy):** Uses the Virtual Memory (VM) system's page fault mechanism. When a processor accesses a page it doesn't have, a page fault is triggered, and the OS fetches the page from the remote node over the network.

Advantage: Easier programming than message-passing (no explicit send/receive). **Disadvantage:** Remote accesses are much slower than local, and maintaining consistency adds overhead.

Q9 Explain SISD, SIMD, MISD, and MIMD models with examples.

These four models form **Flynn's Taxonomy**, classifying architectures by the number of instruction and data streams.

1. SISD (Single Instruction, Single Data):

- **Architecture:** A single processor executes a single instruction stream, operating on a single data stream. Traditional sequential (Von Neumann) computer.
- **Parallelism:** None at the architectural level (though pipelining and superscalar techniques exploit ILP within a single stream).
- **Examples:** Classic Intel Pentium (single-core), older microcontrollers, embedded processors.

2. SIMD (Single Instruction, Multiple Data):

- **Architecture:** A single control unit broadcasts one instruction to multiple processing elements, each operating on different data from its own local memory.
- **Parallelism:** Data-level parallelism (DLP). Ideal for regular, array-based computations.
- **Examples:** NVIDIA GPUs (CUDA cores), Intel SSE/AVX SIMD extensions, Cray vector processors, ILLIAC IV.

3. MISD (Multiple Instruction, Single Data):

- **Architecture:** Multiple processors execute different instructions on the exact same data stream. Extremely rare and mostly theoretical.
- **Parallelism:** Instruction-level parallelism on a single data stream.
- **Examples:** Fault-tolerant systems where multiple redundant processors execute the same program on the same input and vote on the correct output (e.g., Space Shuttle flight control computers). Some argue systolic arrays fit loosely here.

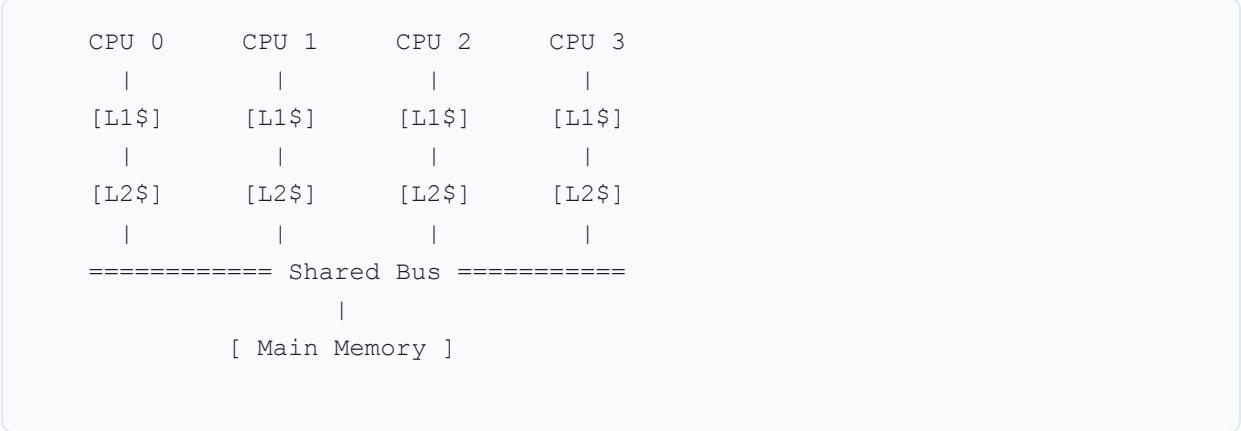
4. MIMD (Multiple Instruction, Multiple Data):

- **Architecture:** Multiple processors, each with its own instruction stream and data stream, executing independently. The most general and flexible class.
- **Parallelism:** Task-level parallelism (TLP). Each processor can run a completely different program.
- **Examples:** Modern multi-core CPUs (Intel Core i9, AMD Ryzen), clusters, cloud data centers, any multi-computer system.

Q10 What is centralized shared memory architecture? Advantages/limitations.

A **Centralized Shared Memory** (also called Symmetric Multiprocessor or SMP) architecture is one where all processors are connected to a single, centralized physical main memory through a shared interconnect (typically a bus or crossbar switch).

Architecture:



Advantages:

1. **Uniform Memory Access (UMA):** All processors have equal access time to all memory, making load balancing and data sharing trivial.
2. **Ease of Programming:** The shared address space allows threads to communicate via shared variables, avoiding the complexity of explicit message passing.
3. **Simple Hardware Coherence:** Snooping protocols (like MESI) work naturally on the shared bus — every cache can observe all transactions.

Limitations:

1. **Bus Bandwidth Bottleneck:** The shared bus has limited bandwidth. As more processors are added, they compete for bus access, saturating the bus beyond ~8-16 processors.
2. **Memory Bandwidth Limitation:** A single memory module has limited ports and bandwidth. Multiple simultaneous requests queue up.
3. **Poor Scalability:** Performance degrades rapidly as the number of processors increases due to contention. Not suitable for systems with more than ~16 processors.

Q11 Differentiate between directory-based and snooping-based cache coherence.

Feature	Snooping-Based	Directory-Based
Mechanism	All cache controllers continuously monitor (snoop) a **shared bus**. Every memory transaction is broadcast to all processors.	A centralized or distributed **directory** keeps track of which caches hold copies of each memory block. Point-to-point messages are sent only to relevant caches.
Communication	**Broadcast:** Every coherence transaction is seen by every processor.	**Unicast/Multicast:** Messages are sent only to the processors that actually hold the data.
Scalability	Poor. Limited by shared bus bandwidth. Practical up to ~8-16 processors.	Excellent. No bus bottleneck. Scales to hundreds or thousands of processors.
Latency	Low for small systems (broadcast is fast on a bus).	Higher per-transaction (requires directory lookup + point-to-point message + response).
Hardware Complexity	Simpler. Cache controllers just need bus snooping logic.	More complex. Requires directory storage (memory overhead proportional to number of cache blocks × number of processors) and a scalable interconnection network.
Examples	Intel desktop multi-core (within a single socket), AMD Bulldozer.	AMD EPYC (Infinity Fabric), Intel Xeon multi-socket (QPI/UPI), SGI Origin.

Key Insight: Snooping is preferred for small, tightly-coupled systems (desktops, single-socket servers) due to its simplicity and low latency. Directory protocols are essential for large-scale systems where bus bandwidth would be a fatal bottleneck.

Q12 Briefly explain interconnection networks. Mention types.

Interconnection Networks are the communication fabric that connects processors, memories, and I/O devices in a multiprocessor or multicomputer system. The choice of network topology directly impacts bandwidth, latency, cost, and scalability.

Key Types:

1. **Shared Bus:** All nodes share a single communication channel. Simplest and cheapest. Bandwidth is shared and limited. Does not scale beyond ~16 nodes. Used in small SMPs.
2. **Crossbar Switch:** A full NxN switch matrix connecting every input to every output simultaneously. Non-blocking (any permutation of connections is possible). Cost grows as $O(N^2)$, limiting scalability. Used in high-end SMPs and network switches.
3. **2D Mesh:** Nodes arranged in a grid. Each node connects to its 4 neighbors (up, down, left, right). Good balance of cost and performance. Diameter = $2(\sqrt{N} - 1)$. Used in many supercomputers and NoC (Network-on-Chip) designs.
4. **Torus:** A mesh with wrap-around connections (edges connect to the opposite side). Reduces diameter and provides more uniform latency. Used in IBM Blue Gene.
5. **Hypercube:** N = 2^k nodes, each connected to k neighbors (neighbors differ by exactly 1 bit in their binary address). Diameter = $\log_2 N$ (very low). Excellent latency but complex wiring for large systems. Used in Connection Machine CM-2.
6. **Omega Network (Multistage):** A logarithmic-depth switching network connecting N inputs to N outputs through $\log_2 N$ stages of 2×2 switches. Cost = O(N log N). Can have blocking. Used in shared-memory multiprocessors.

Q13 Differentiate between strong and weak consistency models.

Strong Consistency (Sequential Consistency):

- **Definition:** The result of any parallel execution is the same as if the operations of all processors were executed in some sequential order, and each processor's operations appear in the order specified by its program.
- **Guarantee:** All processors see all memory writes in the **exact same global order**.
- **Programming:** Easiest to reason about. Programs behave "as expected" by the programmer.
- **Performance:** Most restrictive. The hardware cannot reorder memory operations for performance, severely limiting optimizations like write buffering and out-of-order execution of loads/stores.

Weak Consistency:

- **Definition:** Memory operations can be reordered freely by the hardware for performance. The memory system only guarantees consistency at explicit **synchronization points** (e.g., lock acquire, lock release, barrier).
- **Guarantee:** Before a synchronization operation completes, all previous memory operations by that processor are guaranteed to be visible to other processors. Between synchronization points, anything goes.
- **Programming:** More complex. Programmers must explicitly insert memory barriers/fences at critical points to prevent bugs.
- **Performance:** Much higher. Hardware can aggressively reorder and buffer memory operations, maximizing throughput.

Practical Impact: Most modern processors (ARM, RISC-V) use relaxed/weak models for performance. x86 uses TSO (a middle ground). Programmers use high-level synchronization primitives (mutexes, atomics) provided by languages and libraries, which internally insert the correct barriers.

Q14 What are cluster computers? List their key characteristics.

A **Cluster Computer** is a collection of independent, complete computers (nodes) connected via a high-speed local area network (LAN), working together as a single unified computing resource.

Key Characteristics:

1. **Commodity Hardware:** Built from standard, off-the-shelf servers/PCs. Much cheaper than custom-designed supercomputers or mainframes.
2. **Loosely Coupled:** Each node has its own processor, memory, OS, and storage. Nodes communicate via message passing (e.g., MPI) over a network, not via shared memory.
3. **High Availability:** If one node fails, the others can continue operating. Workload can be redistributed to surviving nodes (fault tolerance).
4. **Scalability:** New nodes can be added incrementally to increase computing power. Scales from a few nodes to thousands.
5. **High-Speed Interconnect:** Nodes are connected via fast networks like Ethernet (10/25/100 Gbps), InfiniBand, or proprietary interconnects for low-latency communication.
6. **Single System Image (SSI):** Middleware software presents the cluster as a single system to users and applications, handling job scheduling, load balancing, and fault management.

Examples: Google's data centers, Amazon AWS EC2 instances, Beowulf clusters, most of the Top500 supercomputers are clusters.

Q15 Compare distributed shared memory and centralized shared memory.

Feature	Centralized Shared Memory (SMP)	Distributed Shared Memory (DSM)
Physical Memory	Single, centralized memory module shared by all processors.	Memory is physically distributed across nodes, but logically shared (single address space).
Access Model	UMA (Uniform Memory Access). All accesses take equal time.	NUMA (Non-Uniform). Local accesses are fast; remote accesses are slow.
Scalability	Poor. Limited by bus/crossbar bandwidth (~2-16 processors).	Excellent. Can scale to hundreds or thousands of processors.
Coherence	Snooping protocols (MESI) on the shared bus. Simple but doesn't scale.	Directory-based protocols. More complex but scales well.
Programming	Easiest. Uniform access means no need to think about data placement.	Moderate. Programmers should optimize data locality for performance (place data near the processor that uses it most).
Cost	Moderate (single motherboard, shared memory).	Higher (multiple nodes, each with own memory, plus interconnect).
Latency	Uniform and low.	Variable. Local = low. Remote = high (10-100x local).
Examples	Intel Xeon single-socket, AMD Ryzen desktop.	AMD EPYC multi-socket, SGI Origin, Intel Xeon multi-socket with QPI.

Q16 Discuss Flynn's taxonomy with examples of modern architectures.

(Refer to Question 9 for detailed explanation. Below are additional modern examples.)

- **SISD:** Any single-threaded application running on one core of a modern CPU. While the CPU itself is MIMD-capable, a single thread uses it as SISD.
- **SIMD:** NVIDIA GPUs running CUDA kernels (thousands of threads executing the same kernel on different data). Intel AVX-512 processing 16 floats with one instruction. Apple Neural Engine for AI inference.
- **MISD:** Triple Modular Redundancy (TMR) systems in aviation and space. Three processors execute the same program on the same input; a voter circuit compares outputs and selects the majority result, tolerating a single processor failure.
- **MIMD:** A modern 16-core AMD Ryzen processor running 16 different threads. A Kubernetes cluster running microservices. A university HPC cluster running different users' simulations simultaneously.

Limitations of Flynn's Taxonomy: It doesn't capture modern hybrid architectures well (e.g., a GPU has SIMD cores organized in MIMD fashion — sometimes called SIMT: Single Instruction, Multiple Threads). It also doesn't distinguish between shared-memory and message-passing MIMD systems.

Q17 Describe architecture and synchronization of centralized shared-memory.

Architecture:

- Multiple CPUs (2-16), each with private L1 and L2 caches.
- All CPUs connect to a shared bus or crossbar interconnect.
- A single, centralized main memory is accessible by all CPUs through the shared interconnect.
- A shared I/O subsystem is also connected to the bus.

Synchronization Mechanisms:

1. **Atomic Instructions (Hardware Level):**
   - **Test-and-Set (TAS):** Atomically reads a memory location, returns its old value, and sets it to 1. Used to implement spin locks.
   - **Compare-and-Swap (CAS):** Atomically compares a memory location to an expected value; if they match, writes a new value. Foundation for lock-free data structures.
   - **Load-Linked / Store-Conditional (LL/SC):** LL reads a value. SC writes a new value only if no other processor has written to that location since the LL. More flexible than TAS.
2. **Spin Locks (Software using atomic instructions):** A thread repeatedly "spins" (loops) checking a lock variable using TAS until the lock is free. Efficient for short critical sections but wastes CPU cycles while spinning.
3. **Barriers:** All threads must reach the barrier before any can proceed. Implemented using a shared counter: each arriving thread atomically increments the counter. When the counter equals the number of threads, all are released.

Q18 **Explain different types of interconnection networks with diagrams and pros/cons.**

1. Shared Bus:

```
[P0] [P1] [P2] [P3]
  |    |    |    |
========================= (Single Bus)
          |
     [Memory]
```

- **Pros:** Simplest, cheapest, easy snooping for coherence.
- **Cons:** Bandwidth shared among all nodes. Fatal bottleneck beyond ~16 nodes.

2. Crossbar Switch:

```
    M0  M1  M2  M3
P0  ×   ×   ×   ×
P1  ×   ×   ×   ×
P2  ×   ×   ×   ×
P3  ×   ×   ×   ×
```

- **Pros:** Non-blocking. Any processor can access any memory simultaneously.
- **Cons:** Cost = $O(N^2)$. Impractical for large N.

3. 2D Mesh:

```
P0 — P1 — P2 — P3
|    |    |    |
P4 — P5 — P6 — P7
|    |    |    |
P8 — P9 — P10— P11
```

- **Pros:** Scalable, simple routing, good for nearest-neighbor communication.
- **Cons:** Diameter grows as $O(\sqrt{N})$. Corner nodes have fewer connections.

4. Hypercube (k-dimensional):

```
3D Hypercube (8 nodes):
000 — 001       100 — 101
 |  ×  |    ↔    |  ×  |
010 — 011       110 — 111
```

- **Pros:** Very low diameter = $\log_2 N$. High bisection bandwidth.
- **Cons:** Node degree = $\log_2 N$ (grows with system size). Complex wiring.

5. Torus: Same as Mesh but with wrap-around edges connecting border nodes. Reduces diameter and provides more uniform latency.

Q19 Describe the working of a distributed shared-memory system.

A DSM system provides the abstraction of a single shared address space over physically distributed memories. Here is how it works at a detailed level:

Hardware-Based DSM (e.g., ccNUMA):

1. **Address Mapping:** The global address space is partitioned. Each node "owns" a range of physical addresses corresponding to its local memory.
2. **Local Access:** When CPU A accesses an address in its own local memory range, the access is served directly from local DRAM at full speed (~100 ns).
3. **Remote Access:** When CPU A accesses an address owned by Node B:
   - The memory controller on Node A recognizes the address as remote.
   - It sends a **request message** over the interconnection network to Node B.
   - Node B's memory controller reads the data from its local DRAM and sends a **reply message** back to Node A.
   - Node A's cache loads the data and serves it to the CPU. Total latency: ~300-500 ns (3-5x slower than local).
4. **Directory-Based Coherence:** Each node maintains a directory that tracks which remote caches hold copies of its locally owned memory blocks. When a write occurs, the directory sends invalidation messages to all remote caches holding stale copies.

Software-Based DSM:

Uses the OS Virtual Memory system. Pages are the unit of sharing. When a processor accesses a page not in its local memory, a page fault triggers the OS to fetch the page from the remote node over the network. This is much coarser-grained and higher-latency than hardware DSM, but cheaper to implement.

Q20 Compare cluster computing and traditional parallel architectures. Real-world use case?

Feature	Cluster Computing	Traditional Parallel (SMP/Mainframe)
Coupling	Loosely coupled. Independent nodes with separate OS instances.	Tightly coupled. Shared memory, shared OS.
Communication	Message passing (MPI) over network. Higher latency.	Shared memory variables. Lower latency.
Hardware	Commodity off-the-shelf (COTS) PCs/servers.	Custom-designed, proprietary hardware.
Cost	Much cheaper per FLOP. Incremental scaling.	Expensive. Requires large upfront investment.
Scalability	Excellent. Can scale to thousands of nodes.	Limited by hardware design (bus bandwidth, memory ports).
Fault Tolerance	Good. Individual node failure doesn't crash the system. Job can be rescheduled.	A single hardware failure can bring down the entire system.
Programming	More complex (explicit message passing, data distribution).	Easier (shared memory, implicit communication).

Real-World Use Case:

Google's Search Infrastructure: Google's search engine runs on massive clusters of thousands of commodity servers. When you type a query, it is distributed across hundreds of machines that each search a fraction of the web index. Results are merged and returned in milliseconds. This architecture provides extreme scalability, fault tolerance (failed servers are simply bypassed), and cost efficiency.

Q21 Explain dataflow computers. How do they differ from Von Neumann?

Feature	Von Neumann Architecture	Dataflow Architecture
Execution Model	Control-driven. A Program Counter (PC) sequentially fetches and executes instructions.	Data-driven. Instructions execute when all their input operands (tokens) are available.
Program Counter	Central PC dictates execution order.	No PC. Execution order is determined by data dependencies.
Memory Model	Shared, mutable global memory. Instructions read/write to global variables.	No global memory (in pure dataflow). Data flows through the computation graph as tokens on arcs.
Parallelism	Implicit. Hardware (OoO, superscalar) or compiler must discover parallelism.	Explicit and natural. All instructions with available data fire simultaneously. Maximum parallelism is exposed automatically.
Side Effects	Prevalent. Instructions can modify global state.	None (pure dataflow). Each instruction consumes input tokens and produces output tokens.
Synchronization	Requires explicit locks, barriers for parallel programs.	Implicit. Data availability inherently synchronizes operations.

Dataflow Execution Example:

```
Expression: (A + B) × (C - D)

Von Neumann:          Dataflow Graph:
1. LOAD A                A   B   C   D
2. LOAD B                 \ /     \ /
3. ADD R1, A, B            +       -
4. LOAD C                   \     /
5. LOAD D                    \   /
6. SUB R2, C, D               ×
7. MUL R3, R1, R2           Result
(Sequential: 7 steps)   (Parallel: + and - fire simultaneously → 3 steps)
```

Q22 Describe systolic array architecture. Explain matrix multiplication.

A **Systolic Array** is a network of tightly coupled, simple Processing Elements (PEs) arranged in a regular pattern (typically a 2D grid). Data flows through the array rhythmically, like blood pulsing through the circulatory system (hence "systolic," from the Greek word for heart contraction).

Architecture:

- Each PE performs a simple operation (typically multiply-and-accumulate: MAC).
- PEs are connected to their immediate neighbors in the grid.
- Data enters from the edges and flows through the array in a synchronized, pipelined manner.
- Each PE receives data, processes it, and passes it to its neighbor in the same clock cycle.
- No PE needs to access global memory — data is reused as it flows through.

Matrix Multiplication (C = A × B) on a 3×3 Systolic Array:

1. The array has 3×3 = 9 PEs, each with an accumulator initialized to 0.
2. Elements of Matrix A flow horizontally (left to right) into the rows.
3. Elements of Matrix B flow vertically (top to bottom) into the columns.
4. Elements are staggered (skewed) so they arrive at the correct PE at the correct time.
5. At each clock cycle, each PE: (a) Multiplies the incoming A element by the incoming B element. (b) Adds the product to its accumulator. (c) Passes A to the right and B downward.
6. After 2N-1 cycles (5 cycles for 3×3), all accumulators contain the final result C[i][j].

Key Advantage: Extremely high data reuse. Each element of A and B is read from memory once but used by 3 PEs. This minimizes memory bandwidth requirements — the bottleneck of traditional architectures.

Modern Example: Google's TPU v1 contains a 256×256 systolic array (65,536 PEs) specifically for neural network matrix multiplications.

Q23 What are reduction computer architectures? How do they differ?

Reduction Computer Architectures are based on the principle of **demand-driven computation** (also called lazy evaluation or call-by-need). They execute programs by repeatedly reducing expressions to their simplest form.

How They Work:

1. The program is represented as an **expression tree** (or graph).
2. Execution begins at the **root** (the final output expression).
3. The root demands the values of its sub-expressions.
4. Each sub-expression, in turn, demands the values of its own sub-expressions, recursively.
5. When a leaf node (a constant or input value) is reached, it returns its value immediately.
6. Values propagate upward, and each node reduces (computes) itself when all required inputs arrive.

How They Differ from Other Architectures:

Aspect	Von Neumann	Dataflow	Reduction
Execution Trigger	Program Counter (control-driven)	Data availability (data-driven/eager)	Demand for result (demand-driven/lazy)
Evaluation Strategy	Eager (evaluate everything)	Eager (fire when data arrives)	Lazy (evaluate only when needed)
Unnecessary Computation	May compute unused results	May compute unused results	Never computes unused results
Best For	General purpose	Parallel numeric computation	Functional programming, symbolic computation

HARD QUESTIONS

QH1 Mesh interconnection network with 4×4 nodes. How many links? Latency scaling?

Given: A 2D Mesh network with 4×4 = 16 processing nodes.

Step 1: Count the Links

In a 2D mesh with N rows and M columns:

```
Horizontal Links = N × (M - 1) = 4 × 3 = 12
Vertical Links = (N - 1) × M = 3 × 4 = 12
Total Links = 12 + 12 = 24 bidirectional links
```

Step 2: Latency Scaling (Diameter)

- The **diameter** of a network is the maximum shortest path between any two nodes. It determines the worst-case communication latency.
- In a 2D mesh with N rows and M columns, the diameter is the distance from one corner to the diagonally opposite corner.
- ```
  Diameter = (N - 1) + (M - 1) = 3 + 3 = 6 hops
  ```
- For a general N×N mesh: Diameter = 2(N - 1). This scales as **O(√P)** where P is the total number of nodes, because N = √P.

Conclusion:

- Total links = **24**.
- Maximum latency (diameter) = **6 hops**.
- Latency scales as **O(√P)** — sub-linear, which is reasonably good for scalability. Doubling the number of nodes only increases worst-case latency by ~41% (√2 factor).
  ```

QH2 Cache coherence problem in a 4-processor shared memory system. Hardware solution?

The Problem — Cache Incoherence Scenario:

1. **Initial State:** Variable X = 10 in main memory. No processor has cached it.
2. **Step 1:** Processor P0 reads X. Its cache now holds X = 10. Main memory: X = 10.
3. **Step 2:** Processor P1 reads X. Its cache now also holds X = 10.
4. **Step 3:** Processor P0 writes X = 20 to its local cache (write-back policy). P0's cache: X = 20. Main memory: still X = 10. P1's cache: still X = 10 (STALE!).
5. **Step 4:** Processor P1 reads X from its cache. It gets X = 10 — the **wrong, stale value**!

P0 and P1 now disagree on the value of X. This is a **cache coherence violation**.

Hardware-Based Solution: MESI Snooping Protocol

1. **All caches monitor (snoop) the shared bus.**
2. In Step 1, P0 reads X. Since no other cache has it, P0's cache line is marked **Exclusive (E)**.
3. In Step 2, P1 reads X. P0's cache controller snoops this read on the bus. It intervenes: both P0 and P1 now hold X in the **Shared (S)** state.
4. In Step 3, P0 writes X = 20. Before writing, P0 broadcasts an **Invalidate** message on the bus. P1's cache controller snoops this invalidate and marks its copy of X as **Invalid (I)**. P0's cache line transitions to **Modified (M)** with X = 20.
5. In Step 4, P1 reads X. It finds X is Invalid in its cache (a miss). It requests X on the bus. P0's controller snoops this request, sees it has the Modified copy, **intervenes** to supply X = 20 directly to P1 (and optionally writes it back to memory). Both caches now have X = 20 in the Shared state.

Result: P1 correctly reads X = 20. Coherence is maintained entirely in hardware, transparently to the software.

QH3 Systolic array with 9 PEs performing 3×3 convolution. Draw data flow and compute steps.

Setup: A 3×3 systolic array with 9 PEs performing a 2D convolution of an image with a 3×3 kernel (filter).

Weight-Stationary Design:

- Each of the 9 PEs is pre-loaded with one weight of the 3×3 kernel:

```
PE(0,0)=W00   PE(0,1)=W01   PE(0,2)=W02
PE(1,0)=W10   PE(1,1)=W11   PE(1,2)=W12
PE(2,0)=W20   PE(2,1)=W21   PE(2,2)=W22
```

- Image pixels flow into the array from the left (horizontally) and partial sums flow downward (vertically).

Data Flow per Clock Cycle:

1. Each PE receives an image pixel from its left neighbor.
2. It multiplies the pixel by its stored weight: `product = pixel × weight`.
3. It adds the product to the partial sum arriving from the PE above: `partial_sum_out = partial_sum_in + product`.
4. It passes the pixel to the right neighbor and the new partial sum downward.

Execution Steps for an N×N Image:

- **Startup (Pipeline Fill):** The first few cycles feed skewed pixel data into the array edges.
- **Steady State:** After the pipeline fills (after about 2×3 - 1 = 5 cycles for a 3×3 array), one output pixel emerges from the bottom row every clock cycle.
- **Total Cycles for NxN image:** Approximately N^2 + startup cycles. For a 3×3 kernel on an N×N image, it takes about N^2 + 4 steps.
- **Scalar Comparison:** A scalar processor would need $N^2 \times 9$ multiply-accumulate operations (9 MACs per output pixel). The systolic array achieves this in ~N^2 steps with 9 PEs, a ~9× speedup.